



***Reporte de Despliegue con Kubernetes:
Proyecto final AutoLife***

Integrantes:

*Levir Heladio Hernandez Suarez
Cristian Alexander Malagon Ruiz
David Santiago Morantes Duarte
Luis Andres Gonzalez Corzo*

*Ingeniería de Software 3
Grupo C1*

***Escuela de ingeniería de Sistemas e Informática
Universidad Industrial de Santander***

2 de Diciembre de 2024

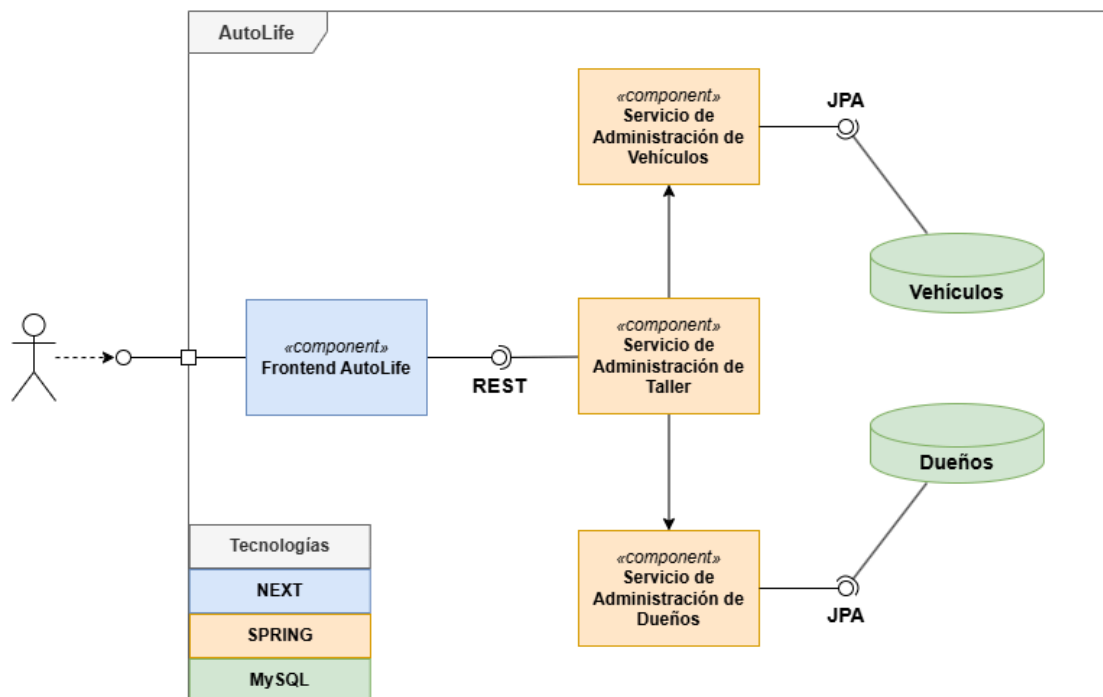
1. Contextualización del Proyecto

Descripción General de la Aplicación

La aplicación **AutoLife** tiene como propósito principal centralizar y automatizar la gestión de información en un entorno de talleres o servicios relacionados con vehículos. Está diseñada para manejar eficientemente los siguientes aspectos:

- 1. Gestión de Dueños:** Registrar, actualizar y eliminar información de propietarios de vehículos, asegurando un control preciso de los datos personales.
- 2. Administración de Vehículos:** Gestionar los vehículos asociados a cada dueño, incluyendo su registro, tipos y detalles relevantes.
- 3. Servicios del Taller:** Controlar la disponibilidad, actualización y registro de servicios ofrecidos por el taller.
- 4. Notificaciones Personalizadas:** Permitir la comunicación efectiva entre el taller y los usuarios mediante notificaciones segmentadas por rol.

Diagrama de componentes



2. Objetivos del Informe

El objetivo principal de este proyecto es implementar y evaluar el rendimiento de una aplicación mediante el despliegue de su backend en un clúster de Kubernetes. Para lograrlo, se realizará la configuración del entorno de despliegue, el ajuste de los recursos necesarios y la ejecución de pruebas de rendimiento utilizando herramientas como JMeter. Con base en los resultados obtenidos, se identificarán áreas de mejora y se propondrán optimizaciones para garantizar un desempeño eficiente y estable bajo diferentes cargas de trabajo.

3. Especificaciones del despliegue

En esta sección se presentan las configuraciones utilizadas para el despliegue y la exposición de los componentes principales del backend, como los servicios o los volúmenes persistentes del proyecto, de la base de datos MySQL y la aplicación desarrollada en Spring Boot.

Docker compose:

```
GNU nano 7.2                                docker-compose.yml
services:
  app:
    image: gradle:8.11.1-jdk21
    container_name: gradle-app
    working_dir: /app
    volumes:
      - /home/luis/docker/software3/AutoLifeBE:/app
      - gradle_cache:/home/gradle/.gradle
    command: >
      /bin/bash -c "
      ./gradlew build &&
      java -jar build/libs/*.jar"
    ports:
      - "8080:8080"
    depends_on:
      - mysql
    networks:
      - app-network

  mysql:
    image: mysql:8.0
    container_name: mysql-server
    environment:
      MYSQL_ROOT_PASSWORD: 1234
      MYSQL_DATABASE: AutoLife
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql
      - /home/luis/docker/software3/AutoLifeBE/src/main/resources/static/database.sql:/docker-entrypoint-initdb.d/database.sql
    networks:
      - app-network

volumes:
  gradle_cache:
  mysql_data:

networks:
  app-network:
```

MYSQL Deployment & service:

```
GNU nano 7.2
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
        - name: mysql
          image: mysql:8
          ports:
            - containerPort: 3306
          env:
            - name: MYSQL_ROOT_PASSWORD
              value: "1234"
            - name: MYSQL_DATABASE
              value: "AutoLife"
          volumeMounts:
            - name: mysql-storage
              mountPath: /var/lib/mysql
            - name: init-script
              mountPath: /docker-entrypoint-initdb.d/init.sql
              subPath: init.sql
      volumes:
        - name: mysql-storage
          persistentVolumeClaim:
            claimName: mysql-pvc
        - name: init-script
          configMap:
            name: mysql-init-script
---
apiVersion: v1
kind: Service
metadata:
  name: mysql
spec:
  ports:
    - port: 3306
      targetPort: 3306
  selector:
    app: mysql
```

MYSQL PersistentVolume & PersistentVolumeClaim:

```
GNU nano 7.2
apiVersion: v1
kind: PersistentVolume
metadata:
  name: mysql-pv
spec:
  capacity:
    storage: 2Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  hostPath:
    path: /var/lib/mysql
```

```
GNU nano 7.2
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mysql-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 2Gi
```

SpringBoot App Deployment & service:

```
GNU nano 7.2
apiVersion: apps/v1
kind: Deployment
metadata:
  name: springboot-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: springboot
  template:
    metadata:
      labels:
        app: springboot
    spec:
      containers:
      - name: springboot
        image: luisagzc/spring:lastest
        ports:
        - containerPort: 8080
---
apiVersion: v1
kind: Service
metadata:
  name: springboot-service
spec:
  selector:
    app: springboot
  ports:
  - protocol: TCP
    port: 8080
    targetPort: 8080
  type: NodePort
```

4. Plan de pruebas

Se divide el plan de pruebas en 2 fases, el primero va relacionado a la variación de la cantidad de peticiones a lo largo del montaje en docker, micro k8s de 1 réplica, con 2 réplicas y con 3 réplicas. Para cada una de ellas se varió la cantidad de solicitudes de 50 a 9000 bajo las mismas condiciones para poder evidenciar el flujo de clientes que el backend puede soportar.

En la segunda fase, se realizan 50 solicitudes para cada uno pero variando esta vez el tiempo en milisegundos de cada uno de ellos, desde 1 hasta 1e-9 con el objetivo de evidenciar quién tiene un mejor tiempo de respuesta.

Ahora bien, para cada una de las fases se tendrá en cuenta el siguiente glosario de conceptos:

Samples (Muestras):

Representa el número total de solicitudes (o "muestras") enviadas durante la prueba. Cada muestra puede ser una solicitud HTTP, una transacción, etc. Este número muestra la cantidad total de interacciones que se han probado.

Average (Promedio):

Es el tiempo promedio que tomó procesar todas las muestras o solicitudes. Se calcula como el tiempo total dividido por el número de muestras.

Min (Mínimo):

Es el tiempo más bajo que ha tomado una solicitud en el conjunto de muestras. Representa la respuesta más rápida de todas las solicitudes.

Max (Máximo):

Es el tiempo más alto que ha tomado una solicitud en el conjunto de muestras. Representa la respuesta más lenta de todas las solicitudes.

Std. Dev. (Desviación estándar):

Muestra la dispersión o variabilidad de los tiempos de respuesta. Una desviación estándar alta indica que los tiempos de respuesta varían mucho, mientras que una baja indica que la mayoría de las solicitudes tienen tiempos de respuesta similares.

Error % (Porcentaje de errores):

Representa el porcentaje de muestras que fallaron respecto al total de muestras. Un error puede ser un tiempo de espera que excede un umbral o cualquier otra condición de fallo definida en JMeter. Un valor alto indica problemas de rendimiento o fallos en la aplicación.

Throughput (Rendimiento):

Mide la cantidad de muestras procesadas por unidad de tiempo, típicamente expresado en muestras por segundo o peticiones por segundo. Es una medida de cuántas solicitudes el sistema es capaz de manejar en un segundo, lo que da una idea del rendimiento del sistema.

KB/sec (Kilobytes por segundo):

Indica la cantidad de datos transferidos por segundo. Esto puede ser útil para evaluar el rendimiento de la red y la eficiencia de la transmisión de datos entre el cliente y el servidor.

Avg. Bytes (Bytes promedio):

Es el tamaño promedio de las respuestas recibidas por muestra. Este valor ayuda a entender cuán pesadas son las respuestas, lo que puede afectar el rendimiento.

Latency (Latencia):

Es el tiempo que tarda en iniciarse la respuesta desde que se realiza la solicitud, es decir, el tiempo transcurrido hasta que se empieza a recibir la respuesta. Esta métrica es especialmente importante en sistemas interactivos.

Connect Time (Tiempo de conexión):

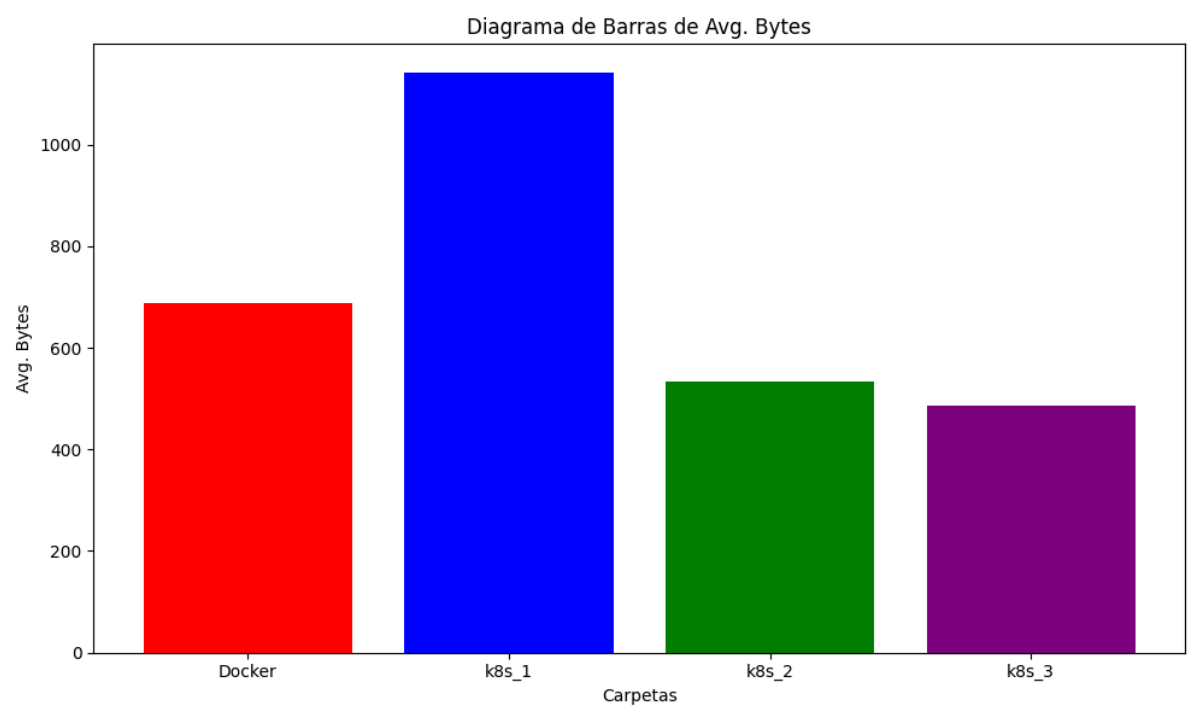
Es el tiempo que tarda en establecerse la conexión entre el cliente (JMeter) y el servidor (el sistema que está siendo probado). Si el

tiempo de conexión es alto, podría indicar problemas con la infraestructura de red o el servidor.

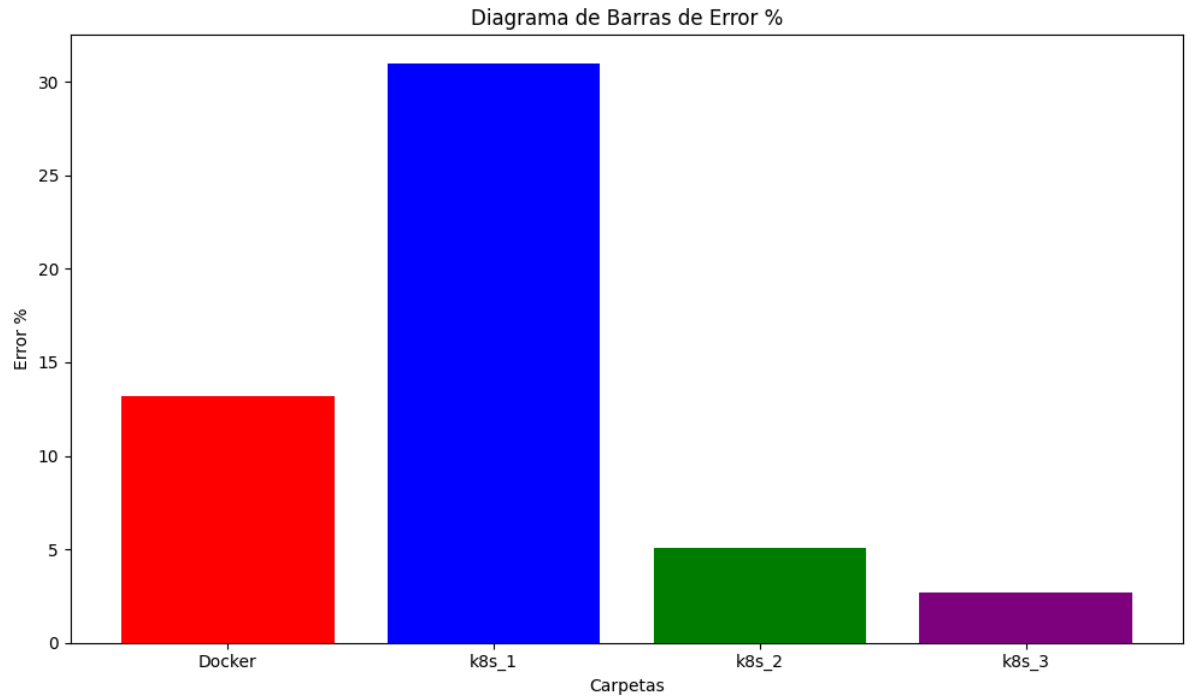
5. Resultados

Solicitudes

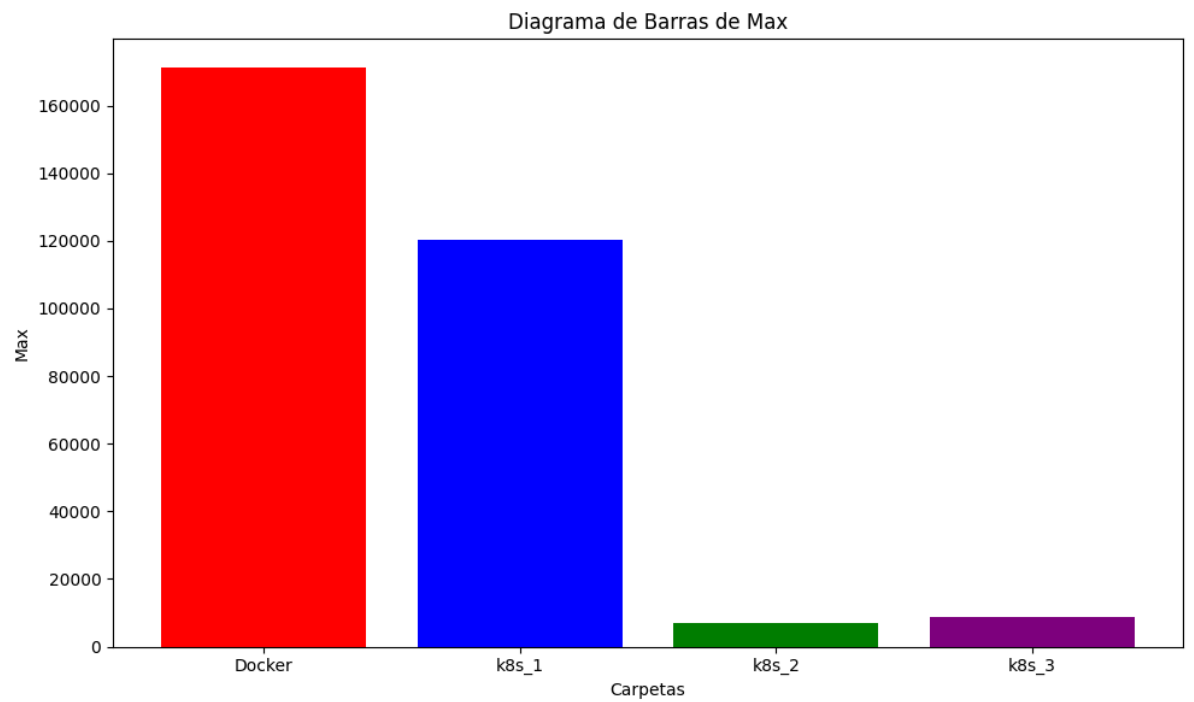
Bytes promedio:



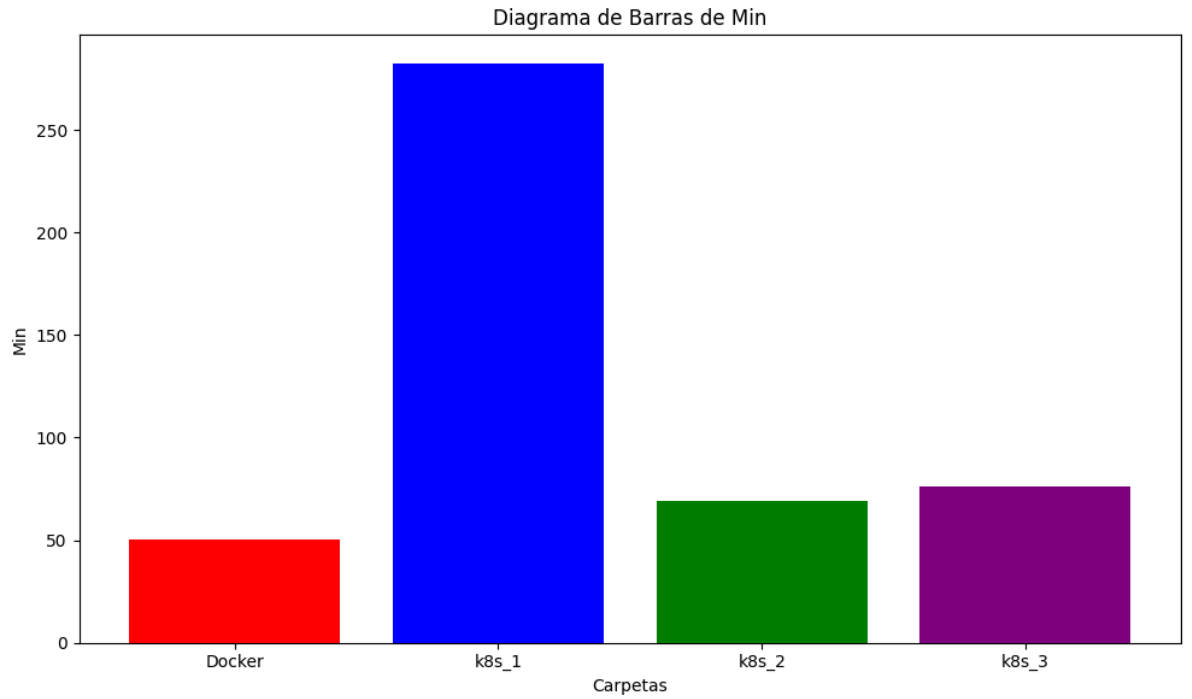
Porcentaje de error:



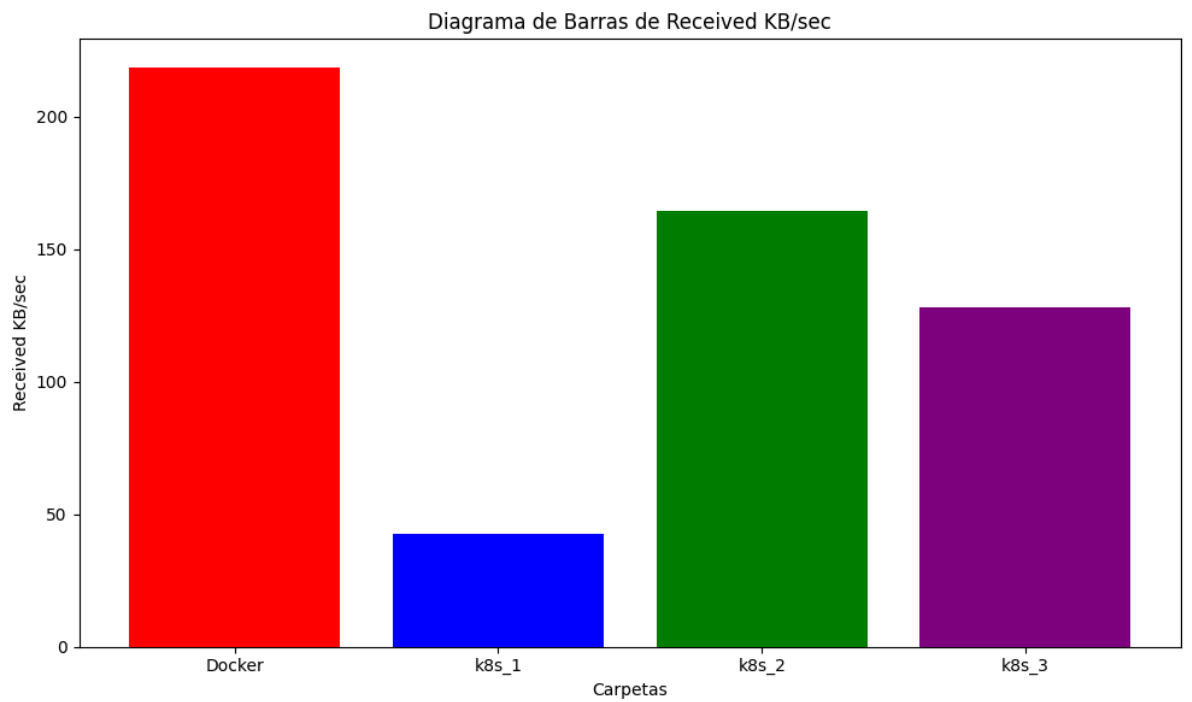
Máximas:



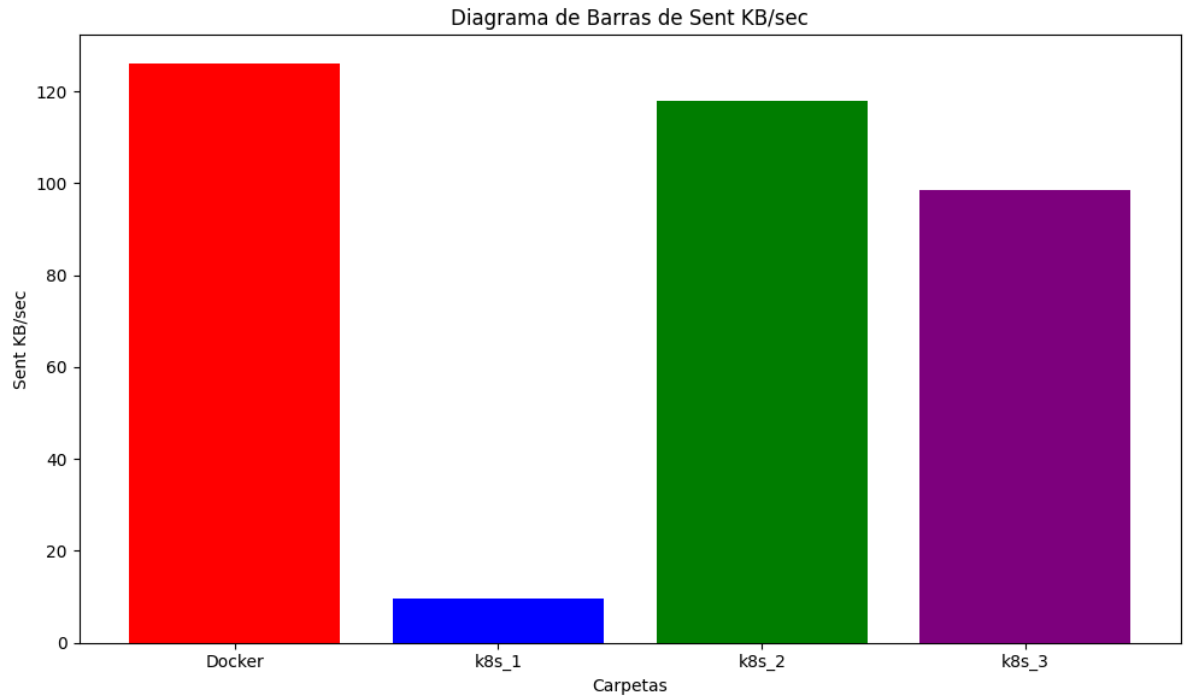
Mínimas:



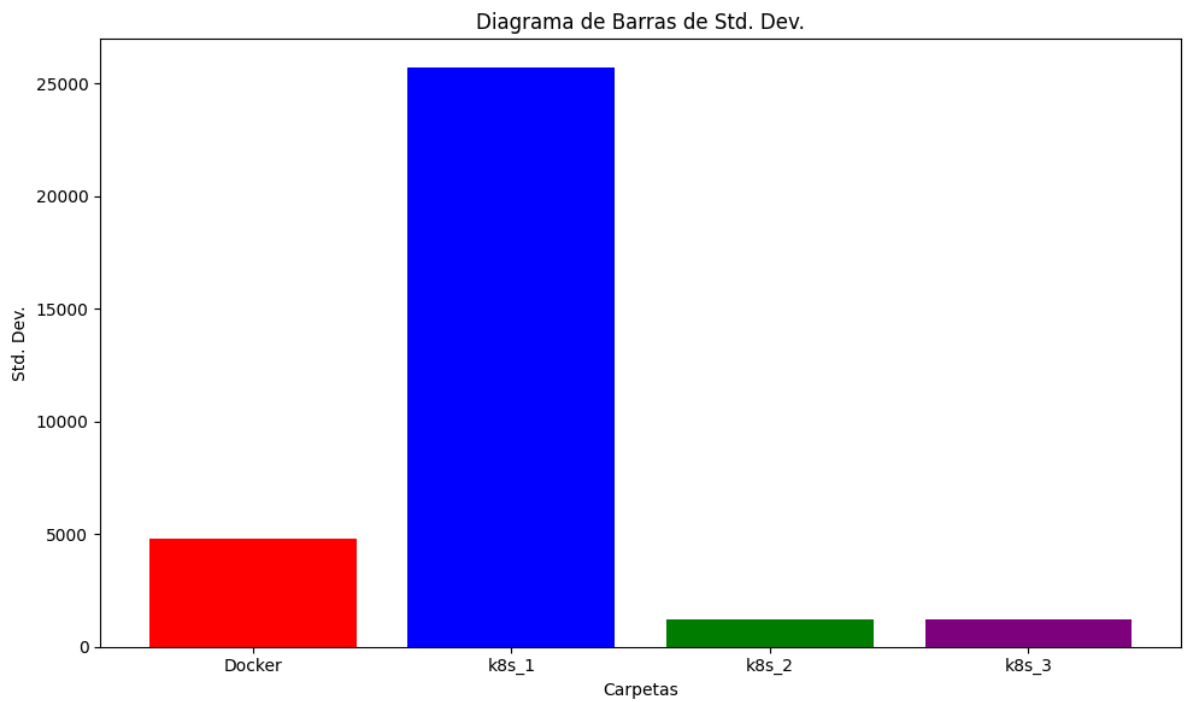
KB recibidos por segundo:



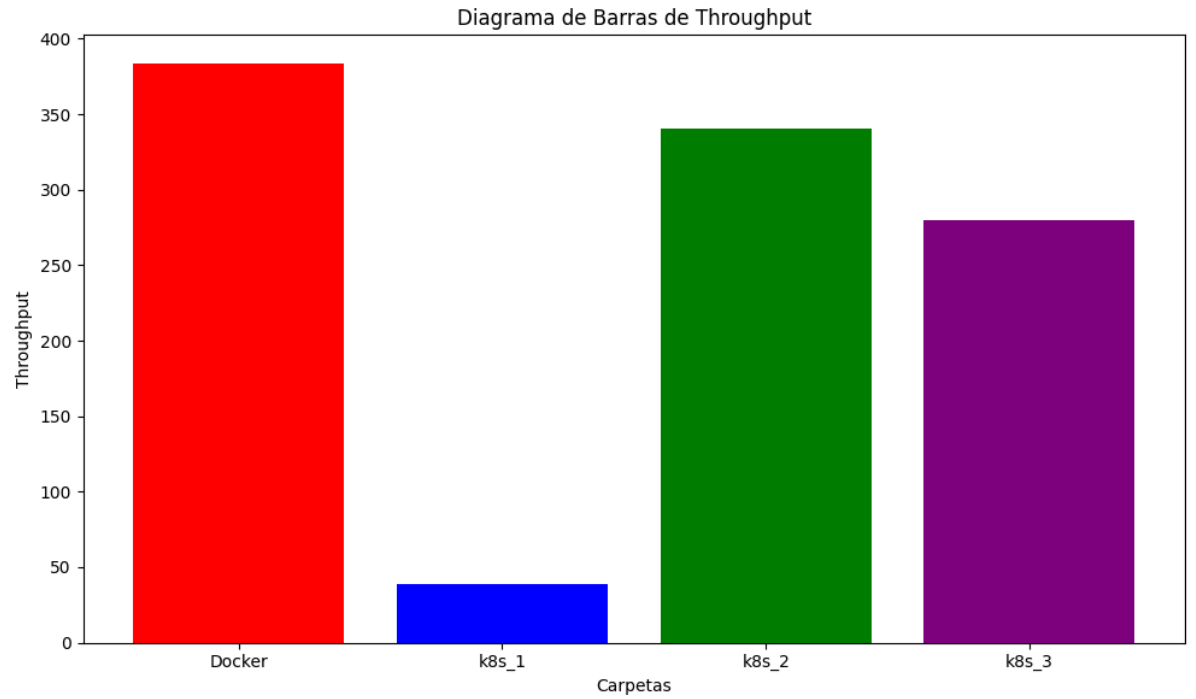
KB enviados por segundo:



Desviación estándar:

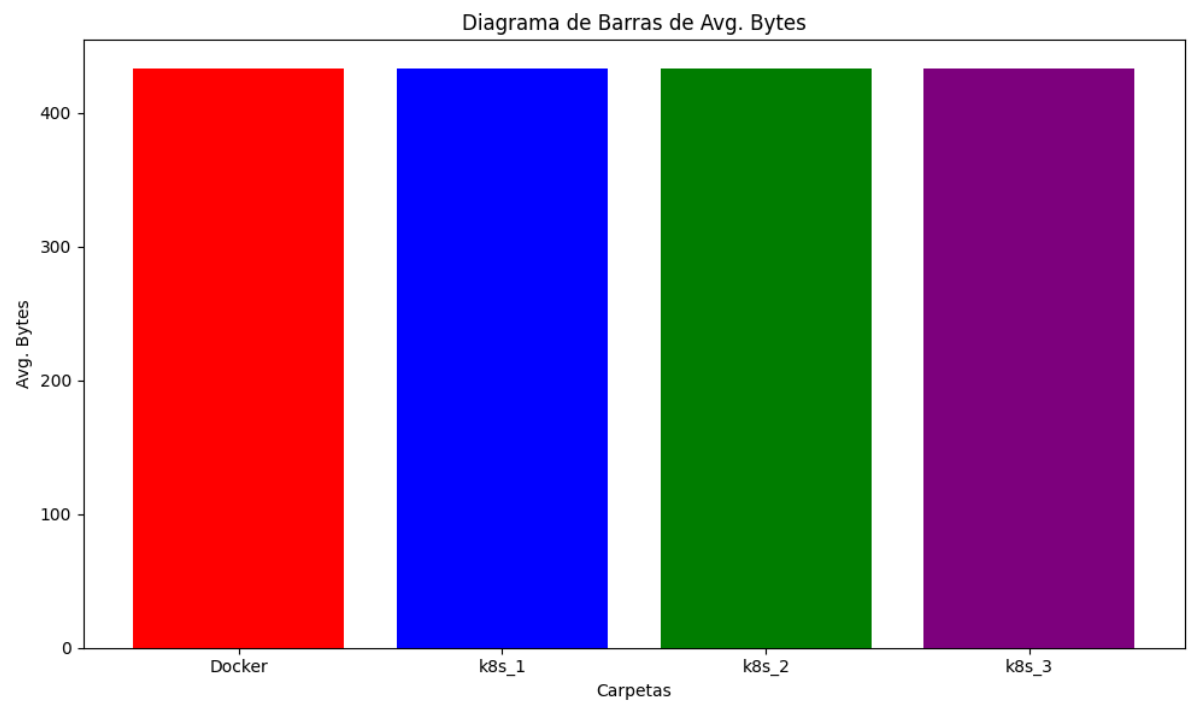


Throughput:

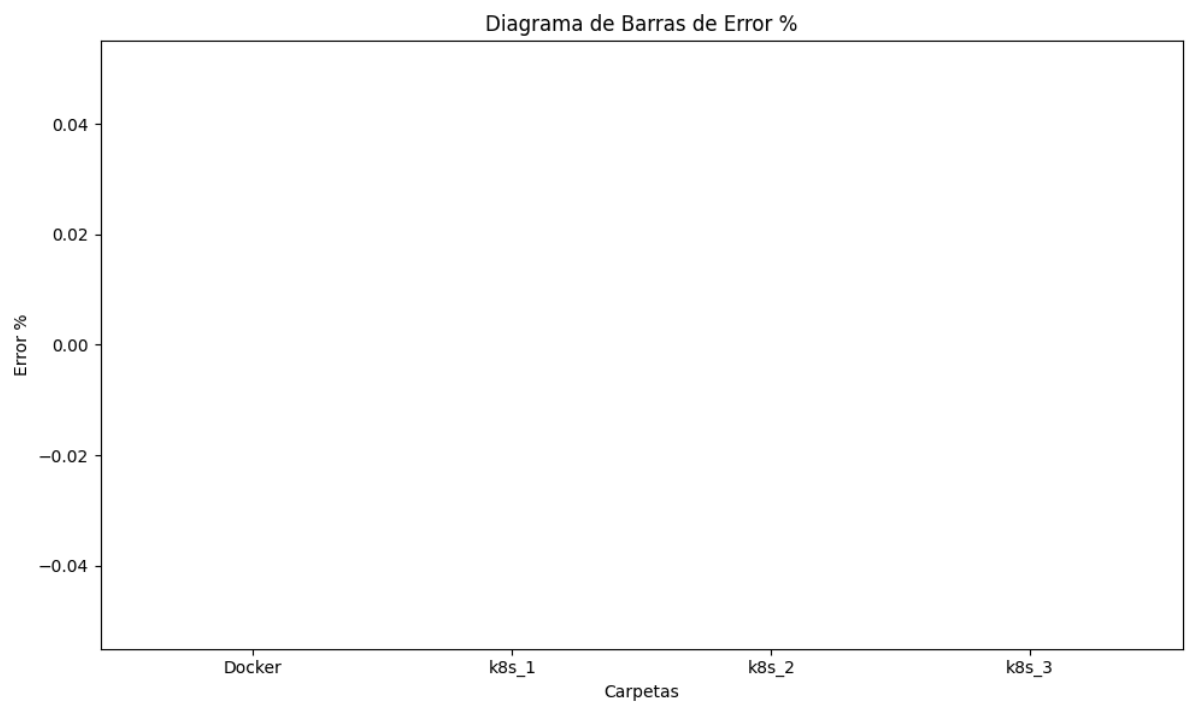


Tiempo

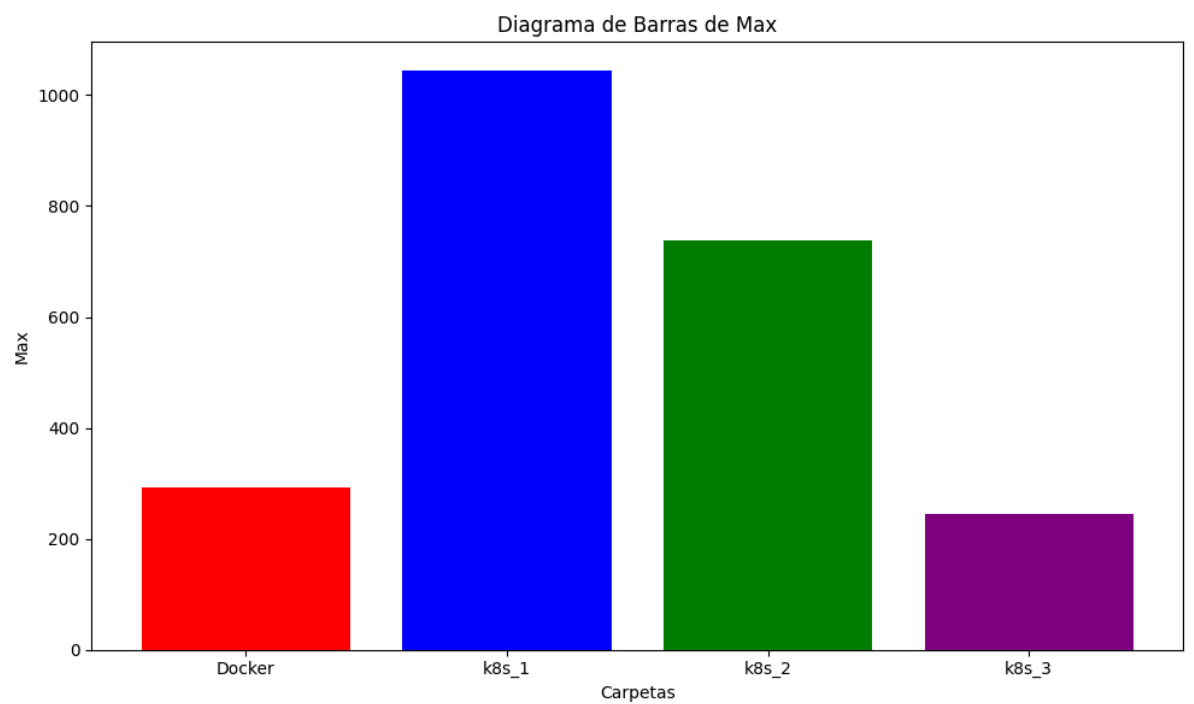
Bytes promedio:



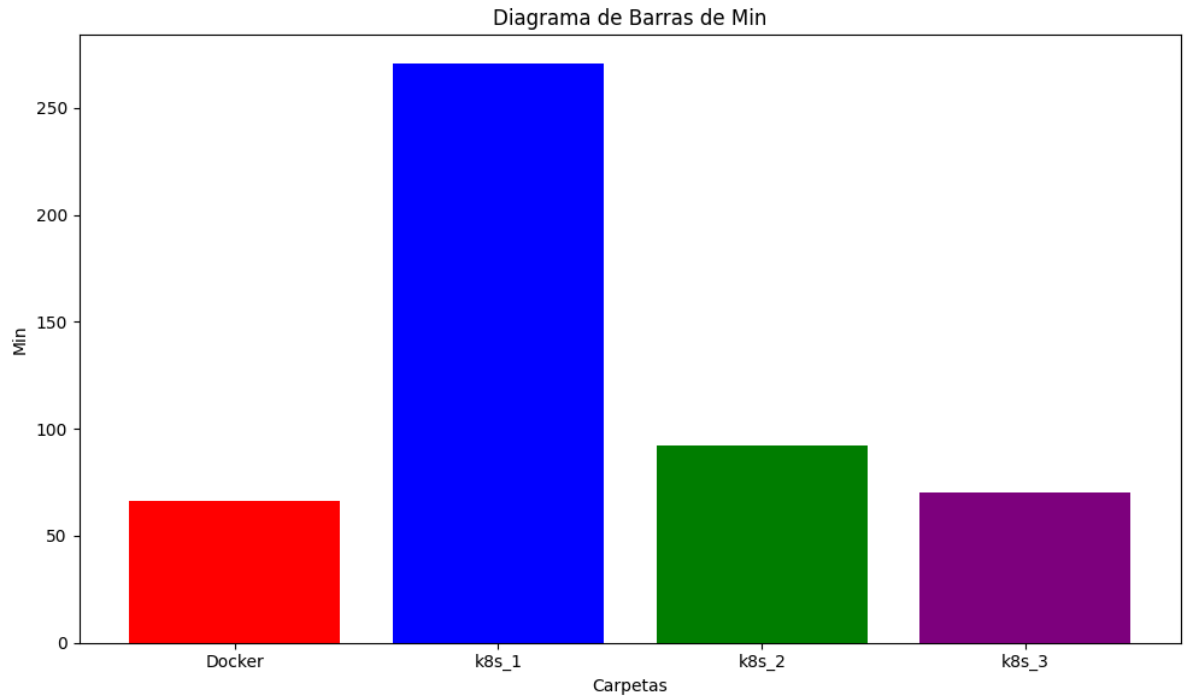
Porcentaje de error:



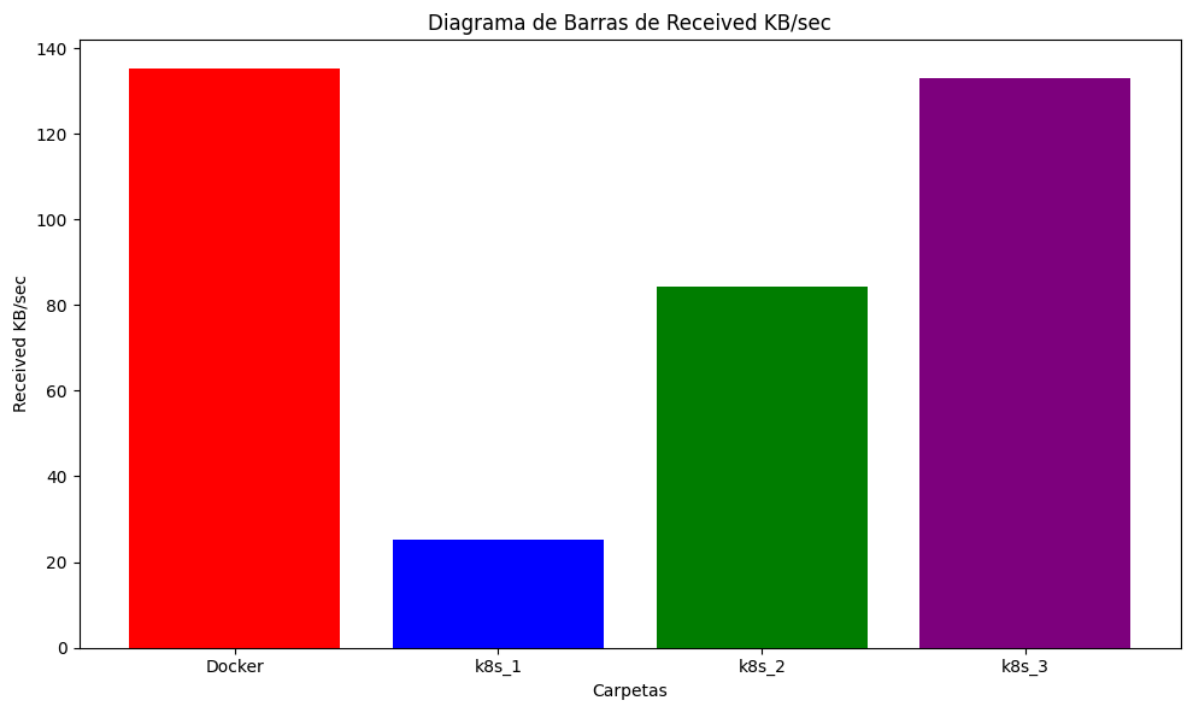
Máximas:



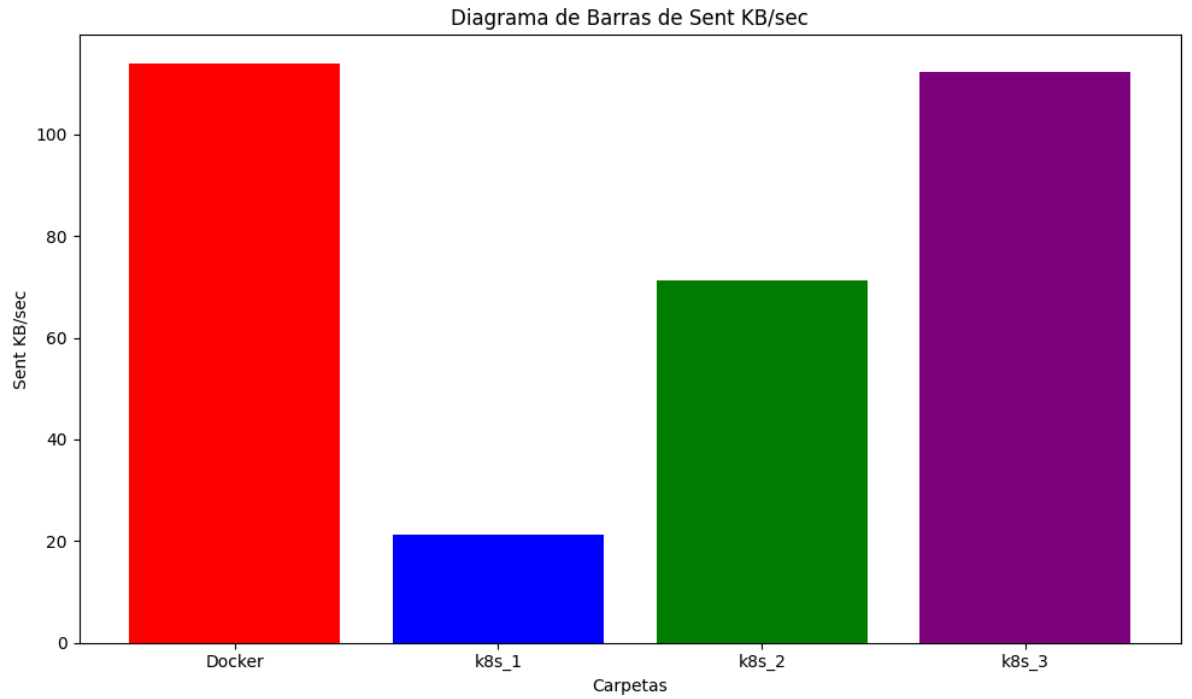
Mínimas:



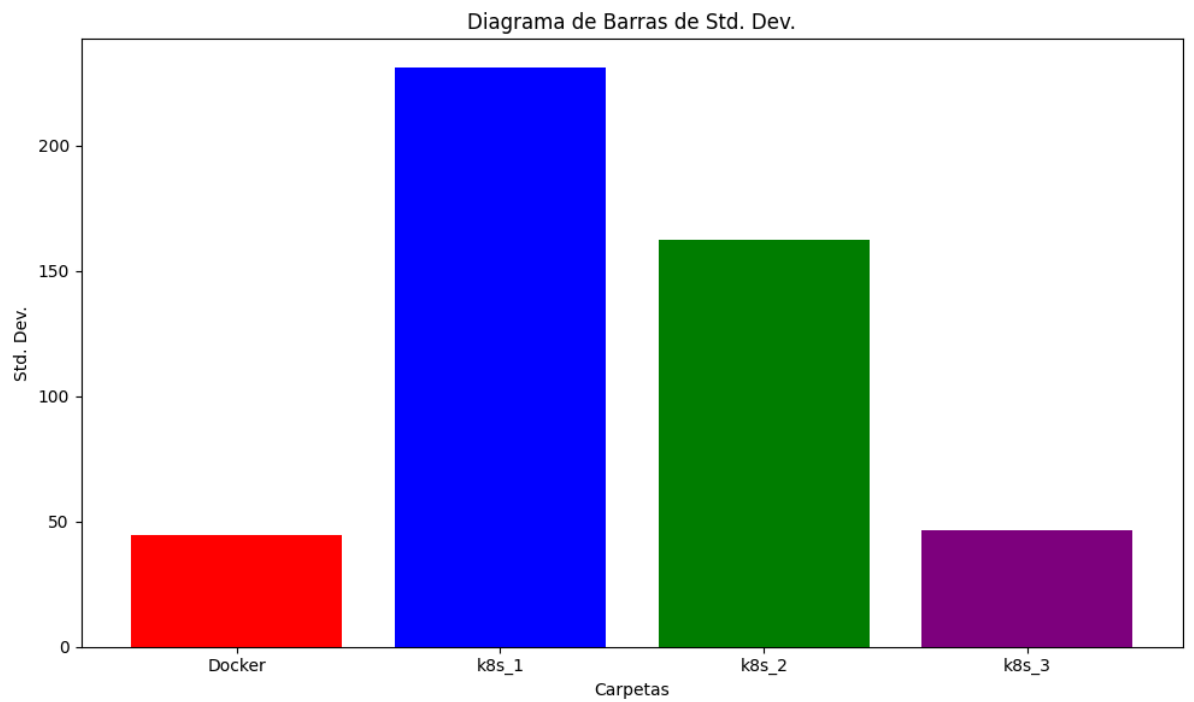
KB recibidos por segundo:



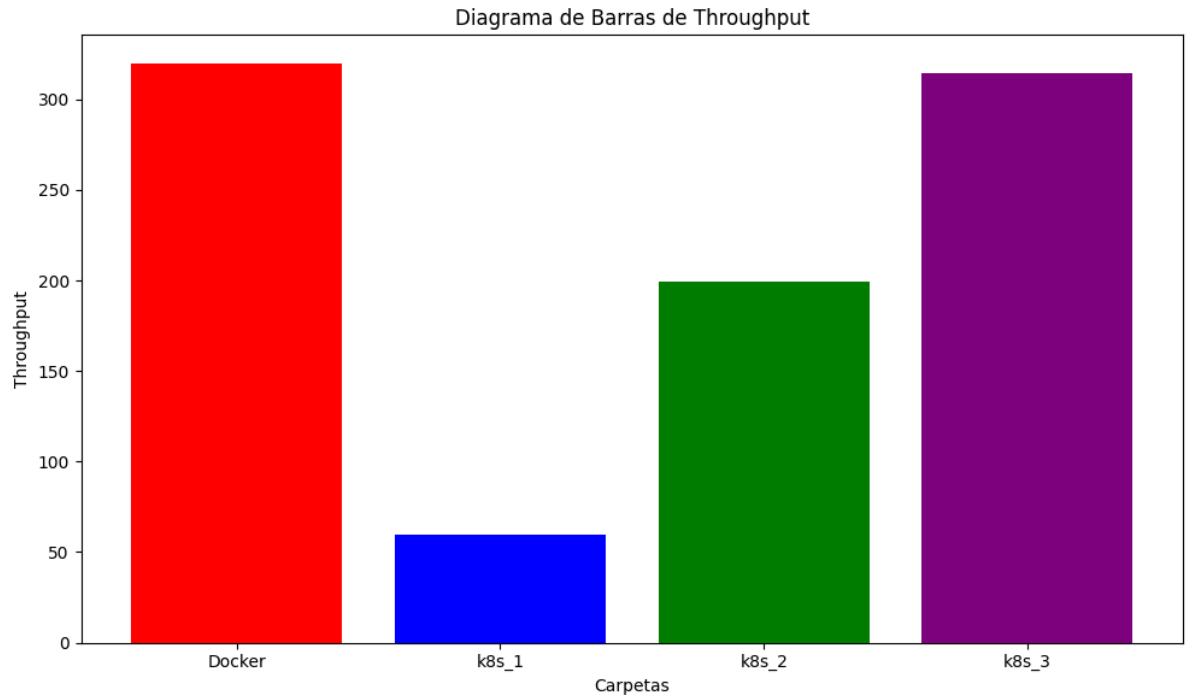
KB enviados por segundo:



Desviación estándar:



Throughput:



6. Conclusiones

Solicitudes

Respecto al número de solicitudes, se puede evidenciar que el uso de microk8s con 1 única réplica presenta un menor rendimiento que lo evidenciado en todos los casos, incluso inferior al de Docker. Esto puede deberse a varios factores que afectan el rendimiento general de la infraestructura, algunos de los cuales incluyen una sobrecarga de MicroK8s en comparación con Docker aunque MicroK8s está diseñado para ser una opción ligera para Kubernetes, sigue siendo una plataforma de orquestación más compleja que Docker. Kubernetes, incluso en su versión más simplificada como MicroK8s, introduce una capa adicional de orquestación que puede generar más latencia y consumo de recursos, especialmente cuando se ejecuta una única réplica. Esto implica que el costo de administración de recursos y la configuración de red en Kubernetes afecta negativamente al rendimiento incurriendo en una pérdida mayor de paquetes.

El hecho de que se esté utilizando solo una réplica con MicroK8s significa que no hay redundancia ni distribución de carga entre varias instancias. En situaciones de alta demanda, como las pruebas que se realizaron, parece ser que se

crea un cuello de botella, ya que todas las solicitudes están siendo manejadas por un único contenedor. Esto contrasta con entornos de producción más robustos que generalmente escalan horizontalmente (más réplicas) para manejar mejor las cargas de trabajo lo que se puede evidenciar en las gráficas al ver un rendimiento muy superior al añadirse un pod adicional.

MicroK8s, aunque es una versión más ligera de Kubernetes, sigue siendo más pesado en términos de uso de recursos que una instancia Docker directa lo que puede estar utilizando CPU y memoria adicionales. Esto puede reducir la cantidad de recursos disponibles para las aplicaciones, afectando el rendimiento de las solicitudes.

En Kubernetes, la gestión de los contenedores, a través de recursos como Pods y Deployments, implica un ciclo de vida adicional (creación, monitoreo, terminación), que podría estar introduciendo una latencia adicional en la administración de las solicitudes.

Tiempo de respuesta

En cuanto al tiempo de respuesta, aunque todos recibieron los paquetes de igual forma, se puede ver un rendimiento igual de bueno entre Kubernetes (K8s) y Docker. Sin embargo, K8s con una sola réplica sigue teniendo un rendimiento inferior en todos sus aspectos. En esta ocasión, se piensa que los factores clave que contribuyen a este comportamiento pueden ser varios aunque K8s, en su conjunto, está diseñado para manejar aplicaciones distribuidas a gran escala, al operar con una sola réplica se pierde la ventaja de la orquestación y la distribución de carga que Kubernetes ofrece, por lo que de igual forma se atribuye a una latencia adicional en la respuesta, ya que la gestión de pods, servicios y endpoints añade complejidad al proceso.

En Kubernetes, especialmente con una sola réplica, la asignación y utilización de recursos (como CPU, memoria y almacenamiento) puede no estar tan optimizada por defecto. La capacidad de escalar y distribuir las cargas de trabajo en múltiples nodos o réplicas no está habilitada, lo que significa que la única réplica está gestionando toda la carga de trabajo, posiblemente sobrecargando los recursos disponibles en un solo nodo. En contraste, Docker puede utilizar estos recursos de manera más eficiente sin la sobrecarga de Kubernetes.

Diapositivas (conclusiones resumidas)

Solicitudes:

- **MicroK8s con una réplica única** presenta un rendimiento inferior al de Docker, debido a la sobrecarga de orquestación de Kubernetes, que consume más recursos y genera latencia adicional.
- El uso de solo **una réplica en MicroK8s** crea un cuello de botella, ya que todas las solicitudes son manejadas por un único contenedor, sin distribución de carga ni redundancia.
- MicroK8s, aunque más ligero que Kubernetes tradicional, sigue siendo más pesado que Docker en términos de consumo de recursos, afectando el rendimiento de las solicitudes.

Tiempo de respuesta:

- **Kubernetes y Docker** muestran tiempos de respuesta similares, pero MicroK8s con una réplica única tiene un rendimiento inferior debido a la falta de escalabilidad y la gestión más compleja de pods y servicios.
- La **asignación de recursos** en MicroK8s no está tan optimizada como en Docker, lo que sobrecarga los recursos de un solo nodo, mientras que Docker maneja los recursos de manera más eficiente sin sobrecarga de orquestación.